UConn

School of Business

Lab 2 — Explaining a Model

SHAP · part lecture, part GitHub

OPIM 5512 · Module 2 · Explainable AI

A good score isn't the finish line — tonight the model explains itself.

WHY WE'RE HERE

The deal

You already have a model that predicts New England demand well ($R^2 \approx 0.90$). Tonight you make it say WHY.

One partner builds the GLOBAL view (what drives the model overall), one builds the LOCAL view (why one hour).

Same GitHub loop as Lab 1: branch → commit → push → pull request → review → merge — then merge both views into one report.

First ~15 minutes are lecture (what SHAP is). Then it's hands-on.

Your only code tonight is ONE SHAP line — type it.

You'll approve a partner's explanation of the same model. Approving what you didn't read is how review dies.

SHAP: give every feature a number — in MW

SHAP splits ONE prediction into one number per feature: how many MW it pushed the answer up (+) or down (−) from the average prediction.

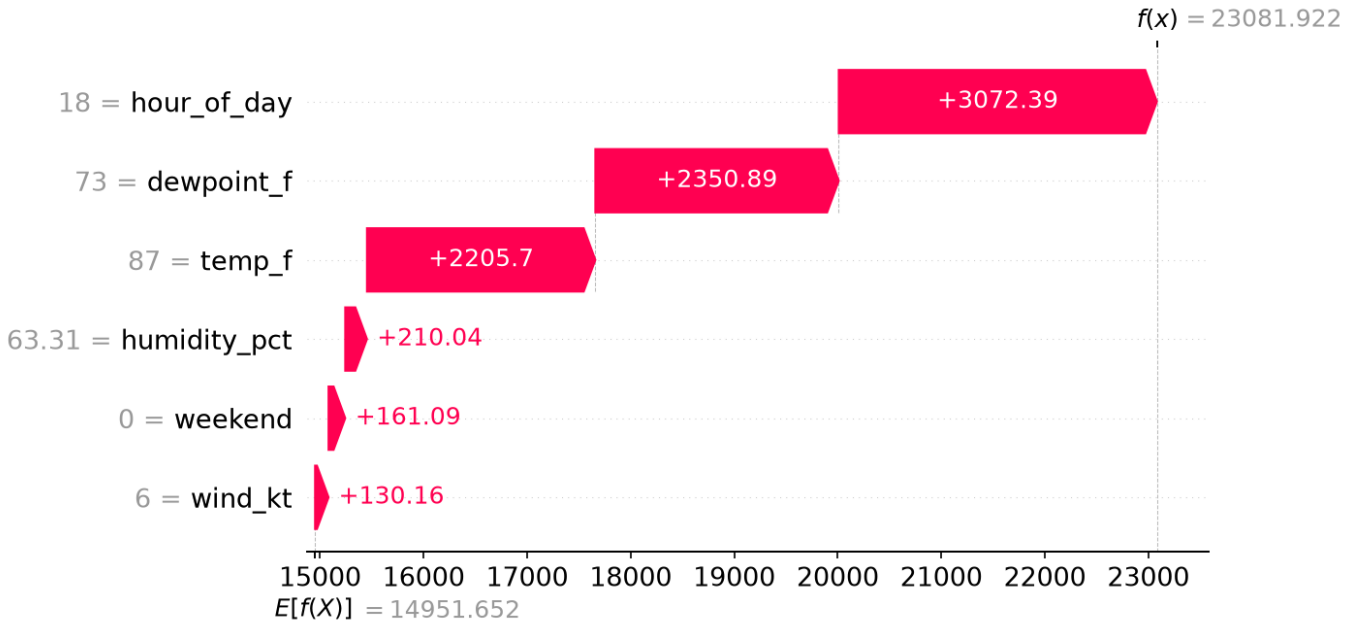
Start at the average, add every feature's push, and you land EXACTLY on the model's answer. It's additive and honest.

average prediction	+	(all the feature pushes)	=	this hour's prediction
14,952 MW	+	+ 8,130 MW	=	23,082 MW

The engine is three lines you DON'T write tonight — the setup cell runs them:

`explainer = shap.TreeExplainer(model)` · `shap_values = explainer(X)`

Local: why THIS one prediction (waterfall)



One hour, one story.

Start at the average (E[f(X)]).

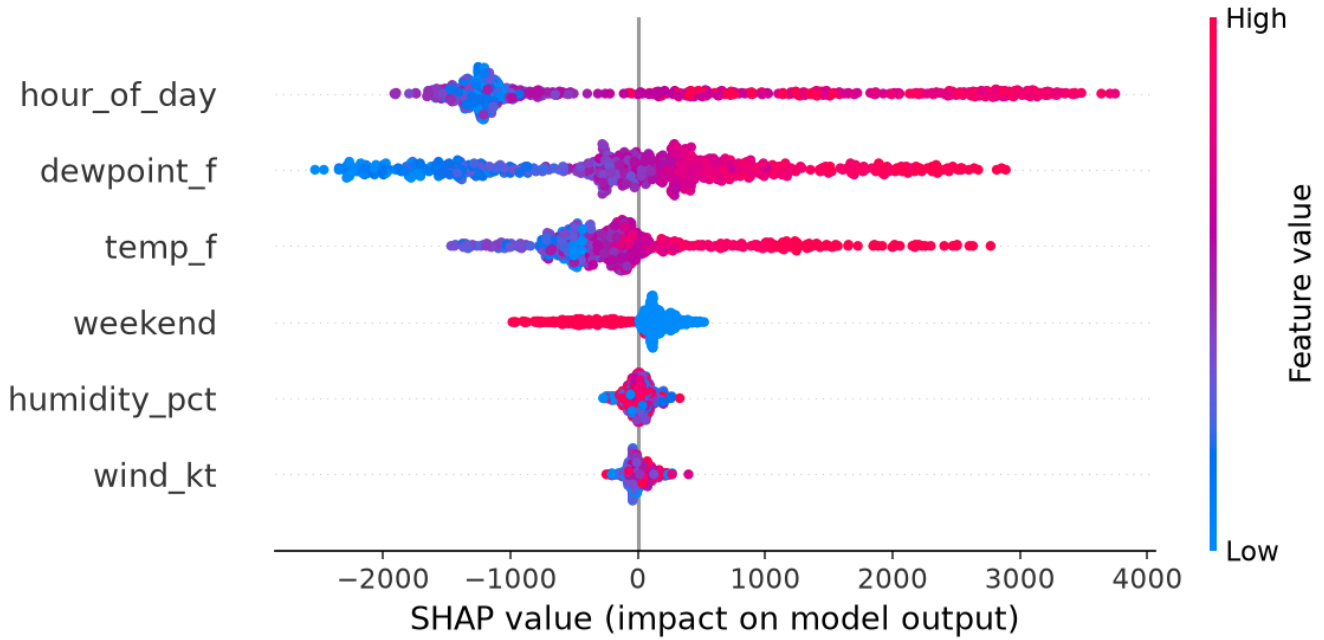
Each bar is a feature's push in MW.

Land on this hour's prediction, f(x).

The peak hour is high BECAUSE it's 6 PM, muggy, and hot.

This is the answer you give a stakeholder.

Global: what drives the model overall (beeswarm)



Stack every hour's explanation.

One dot per hour; color = the feature's value (red high, blue low).

Left/right = push in MW.

Read top to bottom: hour_of_day, dewpoint, temp do the work.

High values (red) push demand UP.

Global = what it leans on. Local = one story.

Stack check & roles

GitHub account + GitHub Desktop installed and signed in

Google Colab opens in your browser

You did Lab 1 — same branch → PR → review → merge loop tonight

Pick roles now — Partner A = GLOBAL (beeswarm), Partner B = LOCAL (waterfall)

Have the instructions open beside Colab — every click is in there, in order

Odd one out pairs with Dave. Online / solo? Two accounts, or pair over Teams.

Make the shared repo — from the template

1 · Pair up — Partner A (global) / Partner B (local)

2 · Partner A: template repo → Use this template → new PUBLIC repo, owner = you

3 · Partner A: Settings → Collaborators → add B → B accepts the invite

4 · Partner A: Settings → Rules → ruleset on main: require a PR + 1 approval

5 · Both: clone the repo ONCE in GitHub Desktop

6 · Each: New branch (dev-global / dev-local) → Publish branch

The template holds the data, both notebooks, images/, README (with the lecture) and REPORT. You build nothing by hand.

Run it, write one SHAP line, ship it

7 · Colab → File → Open notebook → GitHub tab → your repo URL → open your notebook

8 · Runtime → Run all. Setup installs SHAP, fits the model (note R^2), builds shap_values + one free plot

9 · In the TODO cell: A → beeswarm → shap_global.png ; B → waterfall for the peak hour → shap_local.png

10 · Look at your plot. Run the download cell → two PNGs land in Downloads

11 · File → Save the notebook → your repo · your dev- branch · same path

12 · Desktop → Show in Explorer → drag the PNGs into images/ → Commit → Push

One line: shap.plots.beeswarm(shap_values) / shap.plots.waterfall(shap_values[i]). Plain Save, not Ctrl+S.

The two-way loop

13 · Compare & pull request → Create → Reviewers → your partner

14 · Open your PARTNER's PR → Files changed → read their SHAP cell, look at the PNG → Approve

15 · Merge both PRs → Delete branch

16 · Both: Desktop → main → Fetch → Pull — four PNGs and both notebooks are now on your laptop

You can't approve your own PR — that's the gate. Approve only the explanation you actually read.

Do global and local agree?

17 · One screen: REPORT.md → Edit → one sentence per ➜ line (every number gets a unit)

18 · Commit to a new branch 'report' → PR → the other partner approves → Merge

19 · Ahead? Run the joint notebook → shap_dependence.png → images/ → report section 5

20 · Read-out: beeswarm + waterfall on screen — do they tell the SAME story? Check Insights → Network

The key finding: global says what the model leans on; local tells one hour's story. Do they line up?

Don't panic

No module named 'shap'? The setup cell installs it — Runtime → Run all from the top.

SHAP plot errors on shap_values? You skipped the setup cell that builds it. Run all from the top.

Commit early. Once something's committed, it's essentially impossible to lose.

main rejecting your push is the protection working — you're on main when you should be on your branch.

PNG didn't download? The download cell names the missing file — run the cell that makes it, then re-run.

Online / solo students

Pair over Teams if you can — each on your own account, one owns the repo and adds the other.

Otherwise, two accounts (needs a second email) — play both A and B, including approving as the other account.

Simplest solo — one account with Required approvals = 0: branch → PR → merge yourself.

Either way — join the live session on Teams; Friday office hours are highly recommended.

AT 7:30

Definition of done

- ☐ Both partners are collaborators; branch protection on main
- ☐ images/ has FOUR PNGs with the exact filenames (two per partner)
- ☐ Both notebooks saved back with one SHAP line each (beeswarm / waterfall)
- ☐ REPORT.md — one real sentence under each plot, plus one 'what SHAP can't tell us'
- ☐ ≥3 merged pull requests (one each + the report), branches deleted, both authoring AND reviewing
- ☐ A network graph showing the loop going both ways